UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

Facultad de Ciencias de la Computación

Carrera de Ingeniería de Software

Séptimo semestre, paralelo A

Notas de la presentación: justificación del diseño

Taller de modelado de comunicación entre procesos distribuidos

Código de la actividad: GA-TL-02 - FORM

Escenario 1: AeroAndes Reservas

Aplicaciones Distribuidas (ISR-701)

Docente: Gleiston Cicerón Guerrero Ulloa, PhD

Integrantes

Herrera Barco Humberto Aldair	(zona A)
Toaquiza Zambrano Belinda Betzabeth	(zona B)
Piguave Montaño Nixon Antonio	(zona C)
Emanuel Pino Juliana Romina	(zona D)

Quevedo, Ecuador — 11 de noviembre de 2026

1. Decisiones del grupo

Comunicación entre reservas y pagos

Elección. Se eligió una llamada RPC síncrona de Reservas a Pagos (gRPC sobre HTTP/2). Pagos responde CONFIRMADO, RECHAZADO o DESCONOCIDO, y la reserva queda en estado PENDIENTE_PAGO mientras tanto.

Latencia. La llamada directa evita la espera adicional de pasar por un broker y entrega el resultado del pago en cuanto la pasarela responde. Como la pasarela puede tardar hasta 30 s, el cliente ve el estado PENDIENTE_PAGO mientras espera y no queda bloqueado sin información (R4). RPC se diseñó para ofrecer una semántica simple y una comunicación rápida [1]. En pruebas con microservicios, gRPC tuvo menor tiempo de respuesta que REST y GraphQL [2].

Volumen de mensajes. Es un mensaje corto por cada intento de pago. El enunciado no fija una tasa de pagos, así que se dimensiona con la carga de reservas. gRPC reutiliza una conexión HTTP/2 y soporta varias llamadas simultáneas sobre ella.

Necesidad de respuesta inmediata. Necesita respuesta inmediata: Reservas espera el resultado del pago, y en vuelos los usuarios son sensibles al tiempo de espera. Se descartó RabbitMQ en este enlace porque ordena mejor los reintentos, pero añade un salto y retrasa el resultado.

Tolerancia a fallos. Si la pasarela no responde (F1), la llamada vence al superar el plazo máximo de 30 s (R4). El pago queda en DESCONOCIDO, se libera el asiento y, si el cobro sí ocurrió, se reembolsa como compensación (ver la decisión sobre F1, F2 y F3).

Consistencia. El pago debe ejecutarse a lo sumo una vez. Se logra enviando una clave de idempotencia en cada llamada, de modo que un reintento se detecte como duplicado [3].

Fuentes. [1], [2], [3]

Comunicación entre embarque y reservas, y operación sin conexión

Elección. Se eligió gRPC entre el servidor de embarque y Reservas. Los lectores y quioscos siguen comunicándose con el servidor de embarque por sockets TCP, que es la interacción obligatoria con sockets. El servidor de embarque guarda el manifiesto del vuelo para poder operar sin red.

Latencia. R6 exige responder en menos de 1 s. gRPC y Thrift resultaron más rápidos que REST entre microservicios, gracias a su velocidad y a la compresión de los mensajes [4]. Eso respalda gRPC para validar pases rápidamente, en línea con lo medido en [2]. Se descartó REST entre embarque y reservas por esa mayor latencia, y UDP en los lectores porque un mensaje perdido dejaría al pasajero sin respuesta.

Volumen de mensajes. R6 indica unos 40 lectores enviando mensajes cortos de formato fijo. Es un volumen bajo por lector, que una conexión persistente soporta sin dificultad.

Necesidad de respuesta inmediata. Necesita respuesta inmediata mientras haya red, porque el pase se valida en el momento del abordaje. Sin red, la validación se resuelve localmente y la sincronización queda diferida.

Tolerancia a fallos. Si se corta la red del aeropuerto (F3), el servidor valida contra el manifiesto local, guarda cada abordaje en una cola y los envía a Reservas cuando la conexión vuelve. Esto sigue la estrategia de continuar operando durante la partición y procesar después las acciones encoladas [5].

Consistencia. Se acepta consistencia eventual con riesgo acotado. Brewer señala que los cajeros automáticos eligen disponibilidad con consistencia débil, pero con riesgo limitado [6]. Aquí el riesgo se acota porque el manifiesto local contiene solo los pasajeros del vuelo, y las discrepancias se reconcilian al

reconectar.

Fuentes. [4], [5], [6]

Control de asientos y retención

Elección. Reservas pide la retención al inventario por gRPC. Se eligió un bloqueo corto en la base de datos, aplicado solo al cambiar el estado del asiento, combinado con una retención lógica con vencimiento de 10 minutos. En el diagrama corresponden los mensajes `retenerSiLibre`, `confirmarSiRetencionVigente` y `liberarSiRetenidaPor`.

Latencia. El bloqueo dura lo que tarda una actualización condicional, no los 10 minutos de la compra, por lo que no frena a otros clientes. Mantener un registro bloqueado mientras el usuario completa su compra es riesgoso: si se pierde la comunicación, el registro queda bloqueado. Por eso se recomienda verificar la versión de la fila en lugar de conservar el bloqueo, y el ejemplo típico de ese trabajo es la reserva de pasajes de vuelo [7].

Volumen de mensajes. Todas las retenciones y confirmaciones se dirigen al servicio de inventario, que es el único dueño del estado de los asientos. El enunciado no fija una tasa de reservas por segundo.

Necesidad de respuesta inmediata. Necesita respuesta inmediata: retener un asiento o rechazar la solicitud debe resolverse en el momento, para que el cliente sepa si pudo reservar.

Tolerancia a fallos. Si el cliente o el pago se caen a mitad de la compra, la retención vence sola a los 10 minutos (R2) y el asiento se libera. Es la idea de un lease, un derecho con plazo en el que las fallas afectan el rendimiento pero no la corrección [8].

Consistencia. Un asiento nunca debe venderse dos veces (R3). Se garantiza con una única escritura atómica y condicional sobre el estado del asiento, que solo avanza si el asiento sigue libre o retenido por la misma reserva. El primero en llegar gana.

Fuentes. [7], [8]

Uso de la caché de disponibilidad

Elección. Se eligió usar la caché solo para las búsquedas de disponibilidad. Retener y confirmar asientos van siempre al servicio de inventario, y este invalida la entrada de caché cuando un asiento se vende o se libera (mensajes `invalidar` del diagrama).

Latencia. R1 exige 1 200 consultas por segundo con el 95 % por debajo de 300 ms. Consultar la base de datos en cada búsqueda no sostendría esa carga, mientras que la caché responde desde memoria. Se descartó consultar siempre la base de datos por esa razón, y también confirmar compras con datos de la caché, porque podría vender dos veces el mismo asiento (R3).

Volumen de mensajes. Se esperan 1 200 consultas por segundo (R1). Las lecturas se sirven desde la caché, y solo las compras llegan al inventario y a la base de datos.

Necesidad de respuesta inmediata. La búsqueda necesita respuesta inmediata, pero tolera datos ligeramente desactualizados: puede mostrar libre un asiento que ya se vendió. Por eso la compra no se apoya en la caché.

Tolerancia a fallos. Si la caché falla, las búsquedas pasan a la base de datos con mayor latencia, pero el sistema sigue funcionando. Nunca se vende un asiento por un dato viejo, porque la compra siempre se valida en el inventario.

Consistencia. La caché ofrece consistencia débil solo para lectura. Expirar entradas por tiempo devuelve documentos desactualizados, mientras que la invalidación mantiene consistencia fuerte con un costo

similar [9]. Con el benchmark de comercio web TPC-W, la invalidación resultó la más adecuada para comercio electrónico [10].

Fuentes. [9], [10]

Respuesta conjunta a los fallos F1, F2 y F3

Elección. Para F1 se usa un timeout con liberación del asiento y compensación. Para F2 se usa una cola durable en RabbitMQ con reintento. Para F3 se usa un manifiesto local en el servidor de embarque con sincronización posterior.

Latencia. Ninguna de las tres respuestas añade espera al camino normal: cada una actúa solo cuando algo falla. El timeout de 30 s de F1 acota la espera del cliente (R4).

Volumen de mensajes. Las confirmaciones de reserva viajan por la cola hacia Notificaciones (R5), un mensaje por reserva confirmada, que debe llegar en menos de 1 minuto después del pago. El enunciado no fija una tasa para ese flujo. RabbitMQ se puede configurar con nodos agrupados y colas replicadas para ganar disponibilidad, aunque eso compite con el rendimiento [11].

Necesidad de respuesta inmediata. F1 es inmediata: el cliente recibe el estado pendiente sin quedar bloqueado. F2 y F3 son diferidas: la notificación se entrega cuando el servicio vuelve, y los abordajes se envían al recuperar la red.

Tolerancia a fallos. F1: una transacción larga se divide en pasos y, si uno falla, se ejecutan transacciones compensatorias para deshacer los anteriores [12]. En microservicios este patrón no aísla las lecturas de estados incompletos [13], por eso la reserva permanece en PENDIENTE_PAGO y no se muestra como vendida. F2: la confirmación espera en la cola y se reintenta hasta que Notificaciones responde. F3: se aplica lo descrito en la decisión de embarque.

Consistencia. F1 exige que el cobro ocurra una sola vez. Lograrlo ante caídas es difícil, porque cliente y servidor deben registrar sus acciones en almacenamiento estable [14]. F2 y F3 aceptan consistencia eventual, con las garantías y límites que describen [15].

Fuentes. [11], [12], [13], [14], [15]

2. Zona A: Clientes y sistemas externos

Redactado por: Herrera Barco Humberto Aldair

Exposición de interfaz perimetral mediante REST/HTTPS hacia clientes y agencias externas

Elección. Se eligió utilizar una interfaz REST sobre HTTPS mediada por un API Gateway para la comunicación externa con clientes web, móviles y sistemas de agencias de viajes, evitando exponer gRPC o contratos binarios directamente fuera de la red corporativa.

Latencia. Conviene en latencia (requisitos R1 y R2) porque la infraestructura HTTP estándar aprovecha el almacenamiento en caché perimetral, el balanceo de carga y la terminación TLS optimizada, reduciendo la complejidad de integración para clientes heterogéneos sin requerir la sobrecarga de traducción de protocolos personalizados en la frontera. Además, estudios recientes sobre la gestión de arquitectura y calidad demuestran que mantener contratos de API estandarizados previene la acumulación de deuda técnica por acoplamiento perimetral [16], [17].

Volumen de mensajes. Se espera un volumen masivo y concurrente de consultas de disponibilidad (lecturas frecuentes) y un tráfico moderado de transacciones de compra (escrituras). El sistema lo

soporta mediante el escalado horizontal independiente del API Gateway y el uso de análisis estricto de componentes para asegurar que los contratos de servicio no introduzcan dependencias cruzadas innecesarias [18].

Necesidad de respuesta inmediata. Sí la necesita de forma inmediata (síncrona) para las consultas de disponibilidad y la recepción inicial de la orden de reserva, permitiendo que el usuario o la agencia externa obtengan una confirmación instantánea del estado de su solicitud.

Tolerancia a fallos. Si la pasarela perimetral o el servicio externo sufre saturación o caídas (F1), el cliente recibe códigos de estado HTTP estándar (como 503 o 504), permitiendo implementar reintentos con retroceso exponencial en el cliente sin propagar fallos en cascada hacia los microservicios internos.

Consistencia. Se requiere consistencia fuerte en el momento crítico de retener el asiento y confirmar la transacción, mientras que las búsquedas perimetrales de disponibilidad toleran consistencia eventual mediante caché.

Fuentes. [16], [17], [18].

Gestión de plazos (timeouts), reintentos e idempotencia ante servicios externos lentos

Elección. Se eligió configurar plazos de espera estrictos (*timeouts*) acotados para las llamadas hacia pasarelas de pago externas, devolviendo un estado pendiente inmediato en la interfaz y utilizando claves de idempotencia por transacción junto con la compensación por Saga (F1).

Latencia. Para cumplir con los requerimientos de tiempo de respuesta (R1), no se puede permitir que una pasarela externa lenta mantenga bloqueados indefinidamente los hilos de ejecución del servicio de Reservas. Cortar la espera síncrona a tiempo optimiza el rendimiento general y previene la propagación de retardos. Asimismo, optimizaciones de ejecución y análisis a nivel de infraestructura y tiempo de ejecución permiten mantener los tiempos de respuesta dentro de los márgenes esperados [19].

Volumen de mensajes. Se procesa una solicitud por cada intento de pago realizado por los usuarios. El sistema soporta este flujo liberando de inmediato las conexiones de red y delegando la verificación asíncrona o la compensación posterior sin agotar el pool de recursos.

Necesidad de respuesta inmediata. La interfaz del cliente necesita mostrar de inmediato un estado de procesamiento pendiente para evitar la incertidumbre o reenvíos duplicados por parte del usuario, mientras que la verificación definitiva con la pasarela se difiere controladamente.

Tolerancia a fallos. Ante un fallo o ausencia de respuesta de la pasarela (F1), la expiración del plazo activa la transacción compensatoria (Saga) para liberar la retención del asiento en el inventario. Si el cliente reintenta la operación, el uso de identificadores de idempotencia previene cargos duplicados, mitigando las interacciones inestables y problemas de latencia externa identificados en la literatura [16].

Consistencia. Se asegura consistencia eventual transaccional entre el sistema de reservas y los sistemas de pago externos mediante el control de estados idempotentes y compensaciones automatizadas.

Fuentes. [16], [19].

Confirmación diferida de correo electrónico mediante colas y consistencia eventual

Elección. Se eligió desacoplar completamente el envío de correos electrónicos de confirmación y boletos mediante el patrón de bandeja de salida (*transactional outbox*) y una cola de mensajería durable hacia el microservicio de Notificaciones, respaldado por una cola de mensajes fallidos (*DLQ*).

Latencia. R2 establece pautas temporales estrictas para la confirmación de la reserva. Realizar un envío de correo electrónico de forma síncrona añadiría una latencia innecesaria y dependiente de servidores

SMTP externos en la ruta crítica de compra. Extraer esta tarea acelera notablemente la respuesta visual al pasajero.

Volumen de mensajes. Se genera exactamente un evento de notificación por cada reserva confirmada con éxito. El bróker de mensajes absorbe las ráfagas de transacciones y alimenta al servicio de Notificaciones de forma regulada, protegiendo las bases de datos transaccionales de sobrecargas innecesarias y asegurando la evolvibilidad a largo plazo del sistema [20].

Necesidad de respuesta inmediata. El envío del correo debe ser completamente diferido: el pasajero visualiza el resultado y su código de confirmación de manera instantánea en pantalla, y el correo electrónico llega a su bandeja de entrada segundos o minutos después sin alterar la validez jurídica ni operativa de la compra.

Tolerancia a fallos. Si el microservicio de Notificaciones o el servidor SMTP externo se encuentran caídos (F2), los eventos de confirmación no se pierden ya que permanecen almacenados de forma segura en la tabla outbox y en el bróker durable. El sistema ejecuta reintentos automatizados con backoff y aísla los errores persistentes sin afectar la experiencia del usuario ni generar deuda técnica por acoplamiento temporal [17].

Consistencia. Se garantiza consistencia eventual con entrega al menos una vez (*at-least-once*), asegurando que el estado del correo electrónico enviado coincida siempre con el registro persistido en el sistema de reservas.

Fuentes. [16], [17], [20].

3. Zona B: Reservas, inventario y datos

Redactado por: Toaquiza Zambrano Belinda Betzabeth

Comunicación entre reservas e inventario

Elección. Se eligió una llamada RPC síncrona con gRPC sobre HTTP/2 entre `reservas` e `inventario`. Por este enlace viajan la consulta de disponibilidad cuando la caché no tiene el dato (mensaje 1.2), la solicitud de retención (mensaje 2.1) y, desde la zona C, la confirmación y la liberación del asiento. Es la interacción RPC obligatoria de la guía. La respuesta va dentro del mismo mensaje (por ejemplo, `resultado := retenerAsiento(...)`) y cada llamada tiene un plazo máximo de espera.

Latencia. R1 exige que el 95 % de las consultas se responda en menos de 300 ms. Solo las búsquedas con fallo de caché llegan al inventario, y para ellas la llamada directa es el camino más corto. En una comparación entre REST, GraphQL y gRPC en microservicios, gRPC tuvo el menor tiempo de respuesta [2], y RPC se concibió para ofrecer una semántica simple y una comunicación rápida [1]. Se descartó REST por esa mayor latencia, y RabbitMQ porque retener un asiento no puede esperar en una cola.

Volumen de mensajes. R1 plantea hasta 1 200 consultas por segundo, pero la caché absorbe las búsquedas con acierto (véase la decisión sobre la caché en la sección 1). El inventario recibe solo los fallos de caché y las operaciones de retención y confirmación, que son una por intento de compra; el enunciado no fija esa tasa. gRPC usa una conexión HTTP/2 y atiende varias llamadas a la vez sobre ella.

Necesidad de respuesta inmediata. Sí la necesita. Reservas debe saber en el momento si el asiento quedó retenido o ya no está disponible para responder al pasajero, y no puede continuar con el pago mientras esa respuesta no llegue.

Tolerancia a fallos. Si el inventario no responde dentro del plazo, la llamada falla y reservas no continúa con el pago: nunca se da por retenido un asiento sin confirmación. Cada llamada lleva `reservaId`, de

modo que un reintento no genera una segunda retención; evitar que una misma petición se ejecute más de una vez es el objetivo de las semánticas de a lo sumo una vez [3]. Si el inventario está caído, las búsquedas con acierto en caché siguen respondiendo (R1), pero no se retiene ni se confirma ningún asiento.

Consistencia. Se necesita consistencia fuerte (R3). El inventario es el único servicio que decide el estado de los asientos, y reservas solo actúa sobre lo que este le responde. No hay copias del estado de un asiento que puedan contradecirse, salvo la caché, que se usa únicamente para consultar.

Fuentes. [1], [2], [3]

Registro de la reserva y su estado en la base de datos

Elección. Reservas guarda cada reserva y su estado en la base de datos de reservas (PostgreSQL sobre TCP) con escrituras síncronas. La reserva se crea en PENDIENTE_PAGO (mensaje 2.2) y luego pasa a CONFIRMADA o a CANCELADA según el resultado del pago y la respuesta del inventario. Si el cobro se aprobó pero el asiento ya no se pudo confirmar, queda en REEMBOLSO_PENDIENTE. Los asientos y las reservas están en la misma base de datos, y cada servicio escribe solo en sus propias tablas.

Latencia. Cada escritura es una transacción corta sobre un solo registro, así que suma poco a la compra y no toca el camino de búsqueda de R1, que se sirve de la caché. La reserva se registra antes de pasar al cobro porque el cliente ve el estado PENDIENTE_PAGO mientras la pasarela responde (R4).

Volumen de mensajes. Se hace una escritura por cada cambio de estado de una reserva, un volumen mucho menor que el de consultas de R1. El enunciado no fija la tasa de compras.

Necesidad de respuesta inmediata. Sí para las escrituras que definen el estado: reservas no debe seguir adelante si no logró registrarlo. En cambio, el envío de la confirmación al pasajero es diferido (véase la decisión siguiente).

Tolerancia a fallos. Si reservas se cae después de retener el asiento y antes de registrar la reserva, nada queda bloqueado: la retención vence sola a los 10 minutos (R2). Es la idea de un *lease*, un derecho con plazo en el que las fallas afectan el rendimiento pero no la corrección [8]. Además, ninguna transacción queda abierta mientras el pasajero paga, porque un registro bloqueado durante la compra queda atrapado si se pierde la comunicación [7]. Si la base de datos no está disponible, no se retiene ni se confirma, porque no se acepta vender un asiento sin dejarlo registrado.

Consistencia. Es consistencia fuerte (R3). El estado del asiento y el de la reserva están en la misma base de datos, y cada cambio es una transacción atómica. El evento de confirmación se guarda en la misma transacción que la reserva. La protección contra la doble venta se explica en la decisión del grupo sobre el control de asientos (sección 1).

Fuentes. [7], [8]

Publicación de la confirmación hacia notificaciones

Elección. Al confirmar la reserva, reservas guarda el evento `ReservaConfirmada` en la base de datos, dentro de la misma transacción. Después, un ciclo (mensaje 4) lee los eventos pendientes (mensaje 4.1), los publica en RabbitMQ por AMQP como mensajes persistentes (mensaje 4.2) y los marca como publicados (mensaje 4.3). Es el patrón de bandeja de salida (*transactional outbox*).

Latencia. R5 pide que la confirmación llegue en menos de 1 minuto después del pago y permite enviarla después de responder al cliente. El ciclo de publicación solo suma su propio intervalo, que debe ser mucho menor que ese minuto. Se descartó publicar directamente en RabbitMQ dentro del flujo de compra, porque haría esperar al cliente por un servicio que no hace falta para responderle.

Volumen de mensajes. Se genera un evento por cada reserva confirmada; el enunciado no fija esa tasa.

Es un volumen mucho menor que el de las consultas de R1.

Necesidad de respuesta inmediata. No la necesita. El pasajero recibe su respuesta cuando la reserva se confirma, y la notificación se entrega después, de forma asíncrona (R5).

Tolerancia a fallos. F2: si notificaciones está caído, la confirmación no se pierde, porque el evento queda en la base de datos hasta publicarse y luego en una cola durable hasta que notificaciones lo procese. Si reservas se cae entre publicar y marcar, el evento se publica otra vez, así que notificaciones debe tolerar duplicados. Lograr exactamente una vez ante caídas es difícil, porque cliente y servidor deben registrar sus acciones en almacenamiento estable [14]; por eso se acepta entrega al menos una vez con un receptor idempotente. RabbitMQ se ha evaluado en arquitecturas agrupadas de alta disponibilidad y tolerantes a fallos [11].

Consistencia. Se acepta consistencia eventual en este flujo: el estado de la reserva ya es consistente en la base de datos, y la notificación puede llegar con retraso, con los límites que eso implica [15].

Fuentes. [11], [14], [15]

4. Zona C: Pagos y notificaciones

Redactado por: Piguave Montaño Nixon Antonio

Procesamiento del pago y recuperación ante una respuesta incierta

Elección. Se eligió una llamada RPC síncrona entre Reservas y Pagos, implementada con gRPC sobre HTTP/2 y TLS. Pagos se comunica con la pasarela externa mediante REST sobre HTTPS y registra cada intento en PostgreSQL. El mensaje `cobrarReserva` devuelve APROBADO, RECHAZADO o PENDIENTE_VERIFICACION. Estos estados describen el resultado del pago; la confirmación del asiento corresponde al inventario.

Latencia. R4 indica que la pasarela puede tardar hasta 30 s. El diagrama establece una espera máxima de 32 s hacia la pasarela y de 35 s para la llamada entre Reservas y Pagos, dejando un margen para procesar la respuesta. Durante ese tiempo, el cliente muestra PROCESANDO_PAGO y mantiene la interfaz activa. Zhu et al. analizan los costos de comunicación RPC en microservicios, incluida la sobrecarga de su pila de protocolos [21]. Usar gRPC no elimina la espera de la pasarela; los plazos elegidos deben validarse al implementar el sistema.

Volumen de mensajes. Se intercambia una solicitud por intento de pago, con datos como reserva, identificador del intento, importe, moneda y token del medio de pago. El escenario no establece una tasa de pagos por segundo: las 1 200 consultas de R1 corresponden a disponibilidad. Las transacciones del registro de pagos deben ser cortas y no mantener bloqueos de base de datos mientras se espera a la pasarela.

Necesidad de respuesta inmediata. Reservas necesita conocer el resultado para continuar con la compra, por lo que se mantiene el RPC acordado por el grupo. La respuesta puede tardar varios segundos, pero el pasajero recibe información del estado de su operación. Mientras el resultado sea incierto, no debe interpretarse el vencimiento de la llamada como un rechazo definitivo ni iniciarse otro cobro independiente para la misma operación.

Tolerancia a fallos. Ante F1, un timeout deja el intento en PENDIENTE_VERIFICACION. Pagos consulta posteriormente su estado mediante la reconciliación prevista cada 10 s en el diagrama. La retención del asiento conserva su vencimiento de 10 minutos según R2; un timeout no provoca por sí solo su liberación inmediata. Si después se comprueba que hubo un cobro y el inventario devuelve NO_CONFIRMABLE, se solicita la anulación o el reembolso. Si la confirmación del inventario sigue en

estado DESCONOCIDO, primero se verifica su resultado para evitar reembolsar una reserva que pudo haberse confirmado.

Consistencia. Se registra el intento antes de solicitar el cobro y se reutiliza `intentoId` como clave de idempotencia, siempre que la pasarela admita este mecanismo. Laigner et al. incluyen una API de pagos idempotente para evitar cobros duplicados tras una recuperación [22]. La compensación también conserva su propio identificador. Navarro analiza las sagas y las transacciones compensatorias para recuperar la consistencia entre servicios [23]. Esta recuperación no sustituye la consistencia fuerte exigida por R3: un pago aprobado solo permite confirmar la reserva si el inventario valida el asiento y la retención correspondiente.

Fuentes. [21], [22], [23]

Envío asíncrono de la confirmación y del pase de abordar

Elección. Se eligió mensajería asíncrona mediante RabbitMQ sobre AMQP/TCP para entregar el evento `ReservaConfirmada` a Notificaciones. Al recibirlo, el servicio consulta a Reservas mediante gRPC sobre HTTP/2 y TLS para obtener los datos de la notificación. Después prepara la confirmación y el pase de abordar, y solicita su envío a un servidor SMTP mediante TCP con TLS. La entrega desde ese servidor hasta el buzón del pasajero es asíncrona.

Latencia. R5 exige que la confirmación llegue en menos de un minuto después del pago, aunque permite enviarla después de responder al cliente. El diagrama fija un máximo de 3 s para consultar los datos y de 5 s para esperar la aceptación del servidor SMTP. Estos tiempos forman parte del presupuesto de entrega, pero no garantizan por sí solos R5: también deben considerarse la espera en la cola y la entrega hasta el buzón.

Volumen de mensajes. Se publica un evento por reserva confirmada. La cola permite absorber aumentos temporales de mensajes sin exigir que Notificaciones los procese al mismo ritmo que se confirman las reservas. Maharjan et al. comparan distintas tecnologías de colas y muestran que su latencia y capacidad dependen de la carga y del escenario [24]. Por ello, RabbitMQ debe evaluarse con la carga prevista, sin asumir que siempre será la alternativa más rápida.

Necesidad de respuesta inmediata. El envío del correo puede ser diferido y no necesita detener la respuesta de la compra. Por esta razón se usa mensajería en este enlace, mientras que Reservas y Pagos conservan RPC. La consulta de datos y la solicitud al servidor SMTP sí esperan una respuesta dentro del procesamiento de cada notificación.

Tolerancia a fallos. Si Notificaciones está caído cuando se confirma la reserva, el evento queda pendiente para procesarse cuando el servicio se recupere, según el mecanismo de conservación descrito en la siguiente decisión. Si falla la consulta a Reservas o el envío SMTP, se aplica la política de reintentos y de mensajes fallidos. La aceptación del servidor SMTP no debe confundirse con la entrega efectiva al buzón del pasajero.

Consistencia. La notificación se genera a partir de `ReservaConfirmada`, no únicamente de un pago aprobado. Con `reservaId`, Notificaciones consulta si la reserva está confirmada y obtiene destinatario, código de reserva, vuelo y asiento antes de preparar el contenido. Laigner et al. destacan la necesidad de considerar el procesamiento de eventos y las restricciones de integridad entre microservicios [22]. En este diseño, si la consulta no confirma la reserva, no se envía el pase y el caso se registra para su revisión.

Fuentes. [22], [24]

Conservación, reintentos y reproceso de notificaciones fallidas

Elección. Para responder a F2 se combina el outbox de Reservas con una cola quorum durable en RabbitMQ, mensajes persistentes, confirmaciones de publicación y ACK manual del consumidor. Notificaciones utiliza un registro persistente en PostgreSQL para controlar cada envío. Los mensajes que no pueden procesarse se enrutan mediante un DLX hacia una cola de fallidos durable y sin TTL. En el diagrama, el DLX utiliza `dead-letter-strategy=at-least-once` y `overflow=reject-publish`.

Latencia. Ante fallos transitorios se permiten tres reintentos con esperas de 5, 10 y 20 s. Esta separación evita repetir inmediatamente una operación que continúa fallando. Sin embargo, los reintentos y el tiempo de recuperación pueden hacer que se supere el minuto exigido por R5. Si una caída impide cumplir ese plazo, el objetivo es conservar la notificación para recuperarla; no se afirma que el plazo pueda garantizarse durante una indisponibilidad prolongada.

Volumen de mensajes. Se identifica cada notificación mediante la combinación del tipo de evento, la reserva y el canal. El registro permite reconocer una entrega repetida y consultar si el correo ya fue aceptado por SMTP. Los errores permanentes o los intentos agotados pasan a la cola de fallidos para evitar un ciclo continuo de procesamiento. La cola principal tiene un límite de reentregas de 20; la de fallidos no descarta mensajes por superar ese contador.

Necesidad de respuesta inmediata. La confirmación de publicación y el ACK del consumidor cumplen funciones distintas. Reservas marca el evento como publicado después de recibir la confirmación del broker y comprobar que no fue devuelto por falta de enrutamiento. Notificaciones envía su ACK después de guardar la aceptación SMTP, o al reconocer un duplicado ya aceptado. No se espera a que el pasajero abra el correo.

Tolerancia a fallos. La reserva confirmada y su evento de outbox deben guardarse en una misma transacción local. Si la publicación no se confirma, el evento continúa pendiente. Si el consumidor pierde la conexión sin enviar ACK, el mensaje puede reentregarse. Un fallo transitorio provoca un rechazo con reencolado; un error permanente o el agotamiento de los reintentos provoca un rechazo sin reencolado y su traslado a la cola de fallidos. Si el registro de envíos no está disponible, no se envía el correo ni se confirma el mensaje. El reproceso de fallidos se realiza después de corregir la causa y contar con autorización de soporte. Se conserva el mensaje en esa cola hasta confirmar su republicación, salvo que el registro demuestre que ya fue aceptado por SMTP.

Consistencia. La entrega al menos una vez puede producir duplicados, por lo que el consumidor debe reconocer los eventos ya procesados [22]. Navarro también señala la necesidad de idempotencia cuando una operación puede recibirse repetidamente [23]. En nuestro diseño se utiliza una clave estable y un registro persistente, con una reclamación temporal del envío para coordinar ejecuciones simultáneas. Aun así, si SMTP acepta el correo y se pierde su respuesta antes de guardar el resultado, un reintento puede generar un envío duplicado. Por ello no se garantiza una entrega exactamente una vez al destinatario.

Fuentes. [22], [23]

5. Zona D: Aeropuerto

Redactado por: Emanuel Pino Juliana Romina

Lectores y quioscos mediante sockets TCP

Elección. Se mantienen sockets TCP persistentes hacia el servidor de embarque. El mensaje 5 envía `solicitudId, vuelo, pase` y devuelve `permitido, motivo`. Los quioscos usan la misma operación como alternativa a los lectores. TCP entrega un flujo ordenado de bytes; la aplicación debe separar los

mensajes mediante un formato fijo o una longitud explícita [25].

Latencia. R6 pide validar en menos de 1 s. Reutilizar la conexión evita abrirla con cada lectura. Ese segundo incluye la red local, la validación y la respuesta; es un objetivo que debe medirse, no una garantía de TCP.

Volumen de mensajes. Se consideran unos 40 lectores (R6), con una solicitud corta y una respuesta por lectura. No se especifican lecturas por segundo ni cantidad de quioscos; no se confunden 40 dispositivos con 40 mensajes por segundo.

Necesidad de respuesta inmediata. La respuesta es inmediata porque el personal necesita saber si permite el abordaje. Si no llega una respuesta válida, el lector no debe mostrar autorización.

Tolerancia a fallos. Si se pierde la conexión TCP, el dispositivo reconecta y reintenta con la misma `solicitudId`. TCP no garantiza que la aplicación haya registrado el abordaje. Ante F3, los lectores continúan si la red local y el servidor funcionan; el manifiesto no resuelve una caída de esa red local.

Consistencia. El servidor centraliza las lecturas de las puertas y comprueba el estado del pase antes de registrar el abordaje. La comprobación y el registro deben ser atómicos para impedir que dos lectores autoricen el mismo pase. Repetir una solicitud devuelve su resultado previo. Liskov y sus colaboradores estudian la detección de mensajes duplicados para lograr ejecución a lo sumo una vez [3]. Ese trabajo respalda la necesidad de controlar duplicados; aquí se conservan identificadores y resultados, sin adoptar su protocolo basado en relojes sincronizados.

Fuentes. [3], [25]

Validación en línea con Reservas mediante gRPC

Elección. Se mantiene gRPC sobre HTTP/2 entre Embarque y Reservas. En 5.1, la operación `validarYRegistrarAbordaje` recibe `solicitudId`, `vuelo`, `pase` y devuelve `permitido`, `motivo`. Se usa una llamada unaria: una solicitud y una respuesta [26], [27]. La autorización incluye el registro para evitar separar ambas acciones en llamadas independientes.

Latencia. Para cumplir R6, la llamada lleva un plazo que deja margen, dentro de 1 s, para la red local y la respuesta al lector. gRPC permite configurar ese plazo, pero no lo fija por defecto [28]. Niswar y sus colaboradores obtuvieron menor tiempo de respuesta con gRPC que con REST y GraphQL en sus pruebas de consulta de datos [2]. La caracterización de protocolos de comunicación de Kumar y sus colaboradores aporta otro antecedente de evaluación en microservicios [4]. Estos estudios orientan la elección, pero no demuestran que AeroAndes cumpla R6: se debe medir su recorrido completo antes de fijar el plazo.

Volumen de mensajes. Cada validación en línea genera una solicitud y una respuesta a Reservas. El servidor reutiliza el canal; la descarga de manifiestos y la sincronización se realizan aparte. Los 1 200 accesos por segundo de R1 corresponden a búsquedas, no a la tasa de embarque.

Necesidad de respuesta inmediata. Se espera la autorización de Reservas antes de responder al lector. Un vencimiento del plazo significa que el resultado es desconocido: la operación pudo ejecutarse aunque la respuesta no llegara.

Tolerancia a fallos. Ante F3 o un plazo vencido, se utiliza el manifiesto previamente descargado y se conserva la misma `solicitudId`. Reservas debe reconocer los reintentos para no registrar otra vez una operación ya aplicada. Si no hay manifiesto válido para ese vuelo, se requiere revisión del personal.

Consistencia. Reservas comprueba y registra una sola vez cada pase del vuelo mediante una operación atómica e idempotente. El abordaje no vende ni libera asientos: R2 y R3 siguen bajo Inventario, y los pagos y avisos de R4 y R5 conservan sus flujos existentes. gRPC transporta la llamada; la consistencia depende de estas reglas de la aplicación.

Fuentes. [2], [4], [26], [27], [28]

Manifiesto local y recuperación de F3

Elección. Antes del embarque, 5.2 descarga de Reservas el manifiesto del vuelo con su versión y 5.2.1 lo guarda localmente. Durante F3, 5.3 valida el pase y registra el abordaje pendiente en almacenamiento local persistente. Al volver la conexión, 5.4 envía lotes por gRPC y 5.4.1 marca solo los registros confirmados. No se introduce RabbitMQ en el aeropuerto.

Latencia. La consulta local evita esperar a la red externa y permite buscar el objetivo de menos de 1 s de R6. La recuperación se ejecuta en segundo plano para no retrasar nuevas lecturas. El tiempo total de sincronización depende de la duración del corte y de los pendientes acumulados.

Volumen de mensajes. Se descarga un manifiesto por vuelo y se actualiza antes de operar sin red. Cada abordaje aceptado sin conexión añade un registro pendiente; al reconectar se agrupan en lotes acotados. No se fija un tamaño ni una tasa que el enunciado no proporciona.

Necesidad de respuesta inmediata. La validación local responde inmediatamente; la actualización de Reservas puede esperar hasta la recuperación de F3. Solo se confirma al lector después de guardar de forma durable el abordaje y su registro pendiente.

Tolerancia a fallos. El manifiesto debe estar disponible antes del corte. La marca de abordado y el pendiente se guardan en una sola transacción local. Si falla el envío o se pierde su respuesta, los registros permanecen y se reenvían con los mismos identificadores; solo se retiran tras la confirmación individual de Reservas.

Consistencia. Se acepta consistencia eventual entre el aeropuerto y Reservas. Bailis y Ghodsi explican que, si dejan de producirse actualizaciones, las copias terminan convergiendo, pero esa garantía no asegura lecturas actuales durante la desconexión [15]. Por eso, al recuperar la red se intercambian los abordajes pendientes y se revisan los conflictos. El manifiesto puede omitir cambios posteriores a su descarga: se limita al vuelo y su versión, se impiden duplicados locales y se envían discrepancias a revisión del personal. En 5.4 Reservas deduplica por solicitud y por pase del vuelo. Esto no garantiza consistencia global durante F3 ni autoriza vender asientos sin conexión.

Fuentes. [15], [26]

6. Declaración de uso de inteligencia artificial

Debido a que no se realizó una presentación presencial en clase, no se redactaron las preguntas previstas para dicha presentación.

Herramienta	Para qué la usamos	Cómo verificamos
Claude	Apoyo para entender el escenario, revisar el diseño y elaborar la plantilla base de `main.tex`.	Revisión de la correspondencia entre los requisitos del escenario, las decisiones de comunicación y el diagrama; comprobación del formato mediante la compilación del documento.

Referencias

[1] A. D. Birrell y B. J. Nelson, «Implementing Remote Procedure Calls,» *ACM Transactions on Computer Systems*, vol. 2, n.º 1, págs. 39-59, 1984. DOI: 10.1145/2080.357392

[2] M. Niswar, R. A. Safruddin, A. Bustamin e I. Aswad, «Performance Evaluation of Microservices Communication with REST, GraphQL, and gRPC,» *International Journal of Electronics and Telecommunications*, vol. 70, n.º 2, págs. 429-436, 2024. DOI: 10.24425/ijet.2024.149562

[3] B. Liskov, L. Shrira y J. Wroclawski, «Efficient At-Most-Once Messages Based on Synchronized Clocks,» *ACM Transactions on Computer Systems*, vol. 9, n.º 2, págs. 125-142, 1991. DOI: 10.1145/103720.103722

[4] P. K. Kumar, R. Agarwal, R. Shivaprasad, D. Sitaram y S. Kalambur, «Performance Characterization of Communication Protocols in Microservice Applications,» en *2021 International Conference on Smart Applications, Communications and Networking (SmartNets)*, IEEE, 2021, págs. 1-5. DOI: 10.1109/SmartNets50376.2021.9555425

[5] P. M. Melliar-Smith y L. E. Moser, «Surviving Network Partitioning,» *Computer*, vol. 31, n.º 3, págs. 62-68, 1998. DOI: 10.1109/2.660191

[6] E. Brewer, «A Certain Freedom: Thoughts on the CAP Theorem,» en *Proceedings of the ACM Symposium on Principles of Distributed Computing (PODC '10)*, ACM, 2010, pág. 335. DOI: 10.1145/1835698.1835701

[7] F. Laux y M. Laiho, «SQL Access Patterns for Optimistic Concurrency Control,» en *2009 Computation World: Future Computing, Service Computation, Cognitive, Adaptive, Content, Patterns*, IEEE, 2009. DOI: 10.1109/ComputationWorld.2009.63

[8] C. G. Gray y D. R. Cheriton, «Leases: An Efficient Fault-Tolerant Mechanism for Distributed File Cache Consistency,» en *Proceedings of the 12th ACM Symposium on Operating Systems Principles (SOSP '89)*, ACM, 1989, págs. 202-210. DOI: 10.1145/74850.74870

[9] P. Cao y C. Liu, «Maintaining Strong Cache Consistency in the World Wide Web,» *IEEE Transactions on Computers*, vol. 47, n.º 4, págs. 445-457, 1998. DOI: 10.1109/12.675713

[10] L. Y. Cao y M. T. Özsu, «Evaluation of Strong Consistency Web Caching Techniques,» *World Wide Web*, vol. 5, n.º 2, págs. 95-123, 2002. DOI: 10.1023/A:1019697023170

[11] M. Rostański, K. Grochla y A. Seman, «Evaluation of Highly Available and Fault-Tolerant Middleware Clustered Architectures Using RabbitMQ,» en *Proceedings of the 2014 Federated Conference on Computer Science and Information Systems (FedCSIS)*, IEEE, 2014, págs. 879-884. DOI: 10.15439/2014F48

[12] H. Garcia-Molina y K. Salem, «Sagas,» en *Proceedings of the 1987 ACM SIGMOD International Conference on Management of Data (SIGMOD '87)*, ACM, 1987, págs. 249-259. DOI: 10.1145/38713.38742

[13] E. Y. Daraghmi, C.-P. Zhang y S.-M. Yuan, «Enhancing Saga Pattern for Distributed Transactions within a Microservices Architecture,» *Applied Sciences*, vol. 12, n.º 12, pág. 6242, 2022. DOI: 10.3390/app12126242

[14] N. R. Ivaki, F. Araujo y R. Barbosa, «A Middleware for Exactly-Once Semantics in Request-Response Interactions,» en *2012 IEEE 18th Pacific Rim International Symposium on Dependable Computing (PRDC)*, IEEE, 2012, págs. 31-40. DOI: 10.1109/PRDC.2012.23

[15] P. Bailis y A. Ghodsi, «Eventual Consistency Today: Limitations, Extensions, and Beyond,» *ACM Queue*, vol. 11, n.º 3, 2013, **artno** 20. DOI: 10.1145/2460276.2462076

[16] T. Cerny, A. S. Abdelfattah, A. A. Maruf, A. Janes y D. Taibi, «Catalog and detection techniques of microservice anti-patterns and bad smells: A tertiary study,» *Journal of Systems and Software*, vol. 206, pág. 111 829, 2023, ISSN: 0164-1212. DOI: https://doi.org/10.1016/j.jss.2023.111829 dirección: https://www.sciencedirect.com/science/article/pii/S0164121223002248

[17] S. S. de Toledo, A. Martini y D. I. Sjøberg, «Identifying architectural technical debt, principal, and interest in microservices: A multiple-case study,» *Journal of Systems and Software*, vol. 177, pág. 110 968, 2021, ISSN: 0164-1212. DOI: https://doi.org/10.1016/j.jss.2021.110968 dirección: https://www.sciencedirect.com/science/article/pii/S0164121221000650

[18] M. Schiewe, J. Curtis, V. Bushong y T. Cerny, «Advancing Static Code Analysis With Language-Agnostic Component Identification,» *IEEE Access*, vol. 10, págs. 30 743-30 761, 2022. DOI: 10.1109/ACCESS.2022.3160485

[19] C. Wimmer, «GraalVM native image: large-scale static analysis for Java (keynote),» en *Proceedings of the 13th ACM SIGPLAN International Workshop on Virtual Machines and Intermediate Languages*, ép. VMIL 2021, Chicago, IL, USA: Association for Computing Machinery, 2021, pág. 3, ISBN: 9781450391092. DOI: 10.1145/3486606.3488075 dirección: https://doi.org/10.1145/3486606.3488075

[20] J. Bogner, J. Fritzsch, S. Wagner y A. Zimmermann, «Industry practices and challenges for the evolvability assurance of microservices,» *Empirical Software Engineering*, vol. 26, n.º 5, pág. 104, jul. de 2021, ISSN: 1573-7616. DOI: 10.1007/s10664-021-09999-9 dirección: https://doi.org/10.1007/s10664-021-09999-9

[21] X. Zhu et al., «Rethinking RPC Communication for Microservices-based Applications,» en *Proceedings of the 2025 Workshop on Hot Topics in Operating Systems*, ép. HotOS '25, Banff, AB, Canada: Association for Computing Machinery, 2025, págs. 158-164, ISBN: 9798400714757. DOI: 10.1145/3713082.3730375 dirección: https://doi.org/10.1145/3713082.3730375

[22] R. Laigner, Z. Zhang, Y. Liu, L. F. Gomes e Y. Zhou, «Online Marketplace: A Benchmark for Data Management in Microservices,» *Proc. ACM Manag. Data*, vol. 3, n.º 1, 2025. DOI: 10.1145/3709653 dirección: https://doi.org/10.1145/3709653

[23] A. Navarro, «Local, Global, and Saga Transactions for SOA Services and Microservices (∗Services),» *IEEE Access*, vol. 13, págs. 80 206-80 246, 2025, ISSN: 2169-3536. DOI: 10.1109/ACCESS.2025.3566230

[24] R. Maharjan, M. S. H. Chy, M. A. Arju y T. Cerny, «Benchmarking Message Queues,» *Telecom*, vol. 4, n.º 2, págs. 298-312, 2023, ISSN: 2673-4001. DOI: 10.3390/telecom4020018 dirección: https://www.mdpi.com/2673-4001/4/2/18

[25] «Transmission Control Protocol (TCP),» RFC Editor, RFC 9293, ago. de 2022. visitado 11 de nov. de 2026. dirección: https://www.rfc-editor.org/rfc/rfc9293

[26] gRPC Authors. «Core concepts, architecture and lifecycle,» visitado 11 de nov. de 2026. dirección: https://grpc.io/docs/what-is-grpc/core-concepts/

[27] gRPC Authors. «About gRPC,» visitado 11 de nov. de 2026. dirección: https://grpc.io/about/

[28] gRPC Authors. «Deadlines,» visitado 11 de nov. de 2026. dirección: https://grpc.io/docs/guides/deadlines/

[29] M. van Steen y A. S. Tanenbaum, *Distributed Systems*, 4.ª ed. Maarten van Steen, 2023, ISBN: 978-90-815406-3-6.

[30] M. Kleppmann, *Designing Data-Intensive Applications, The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*, 1.ª ed. Sebastopol, CA, USA: O'Reilly Media, 2017, ISBN: 978-1-449-37332-0.

[31] S. Newman, *Building Microservices, Designing Fine-Grained Systems*, 2.ª ed. Sebastopol, CA, USA: O'Reilly Media, 2021, ISBN: 978-1-4920-3402-5.

[32] G. Blinowski, A. Ojdowska y A. Przybyłek, «Monolithic vs. Microservice Architecture: A Performance and Scalability Evaluation,» *IEEE Access*, vol. 10, págs. 20 357-20 374, 2022. DOI: 10.1109/ACCESS.2022.3152803

[33] I. Karabey Aksakalli, T. Çelik, A. B. Can y B. Tekinerdoğan, «Deployment and Communication Patterns in Microservice Architectures: A Systematic Literature Review,» *Journal of Systems and Software*, vol. 180, 2021, **artno** 111014. DOI: 10.1016/j.jss.2021.111014